

Project 1: E-commerce Scrapy Crawler - Complete Documentation & Interview Guide

Created by: Kish Siddammanavar

For: Rubick.ai SDE I Application

Date: October 6, 2026

Project Location: C:\Users\RT\PROJECTS\Rubick_AI_Web_Scraping_Projects\project-1-scrapy-ecommerce-crawler

☐ Table of Contents

- [1. Complete Setup & Deployment Commands](#)
- [2. How the Project Works - Architecture & Process](#)
- [3. Use Cases & Applications](#)
- [4. Interview Questions & Answers](#)
- [5. Technical Deep Dive](#)
- [6. Troubleshooting Guide](#)

1. Complete Setup & Deployment Commands

☐ Prerequisites Installation

Install Python 3.8+

```
# Check Python version
python --version

# Should show: Python 3.8 or higher
```

Install MongoDB

```
# Download from: https://www.mongodb.com/try/download/community
# Or using Chocolatey:
choco install mongodb

# Verify installation
Get-ChildItem "C:\Program Files\MongoDB\Server" -Directory
```

☐ Project Setup (Step-by-Step)

Step 1: Navigate to Project Directory

```
cd C:\Users\RT\PROJECTS\Rubick_AI_Web_Scraping_Projects\project-1-scrapy-ecommerce-crawler
```

Step 2: Create Virtual Environment

```
# Create venv
python -m venv venv

# Activate venv (Windows)
venv\Scripts\activate

# You should see (venv) at the start of your prompt
```

Step 3: Install Dependencies

```
# Install all required packages
pip install scrapy pymongo python-dotenv itemadapter

# OR use requirements.txt
pip install -r requirements.txt

# Verify Scrapy installation
scrapy --version
```

Step 4: Start MongoDB Server

```
# Option 1: Start MongoDB manually
& "C:\Program Files\MongoDB\Server\9.0\bin\mongod.exe" --dbpath "C:\data\db"

# Option 2: Start as Windows Service
Start-Service MongoDB

# OR
net start MongoDB

# Keep this terminal open (if using Option 1)
```

Step 5: Verify MongoDB Connection

```
# Test connection
python -c "import pymongo; client = pymongo.MongoClient('mongodb://localhost:27017/', serverSelectionTimeoutMS=2000); client.server_;
```

☐ Deployment & Execution

Run the Spider

```
# Basic run (saves to MongoDB + products.json)
scrapy crawl books_spider

# Run with custom output file
scrapy crawl books_spider -o output.json

# Run with specific log level
scrapy crawl books_spider -L INFO

# Run with limited pages (for testing)
scrapy crawl books_spider -s CLOSESPIDER_PAGECOUNT=5
```

Expected Output

```
2026-10-06 00:30:00 [scrapy.core.engine] INFO: Spider opened
2026-10-06 00:30:01 [books_spider] INFO: Parsing page: http://books.toscrape.com/
2026-10-06 00:30:02 [books_spider] INFO: Scraped product: A Light in the Attic
2026-10-06 00:30:02 [books_spider] DEBUG: Product saved to MongoDB
...
2026-10-06 00:35:00 [scrapy.statscollectors] INFO: Dumping Scrapy stats:
{
  'item_scraped_count': 1000,
  'response_received_count': 50,
  ...
}
2026-10-06 00:35:00 [scrapy.core.engine] INFO: Spider closed (finished)
```

☐ Verify Data Collection

Check MongoDB Data

```
# Count documents
python -c "import pymongo; client = pymongo.MongoClient('mongodb://localhost:27017/'); db = client['ecommerce_db']; count = db.products.count()"

# View sample product
python -c "import pymongo; client = pymongo.MongoClient('mongodb://localhost:27017/'); db = client['ecommerce_db']; product = db.products.find_one()"
```

Check JSON File

```
# Check file exists and size
dir products.json

# View first few lines
Get-Content products.json -Head 20
```

Using MongoDB Compass (GUI)

1. Open MongoDB Compass
2. Connect to: mongodb://localhost:27017
3. Database: ecommerce_db
4. Collection: products
5. Browse and query your data

☐ Re-run & Update

Clean Previous Data

```
# Delete JSON file
Remove-Item products.json

# Clear MongoDB collection
python -c "import pymongo; client = pymongo.MongoClient('mongodb://localhost:27017/'); db = client['ecommerce_db']; db.products.delete_many({})"
```

Run Fresh Scrape

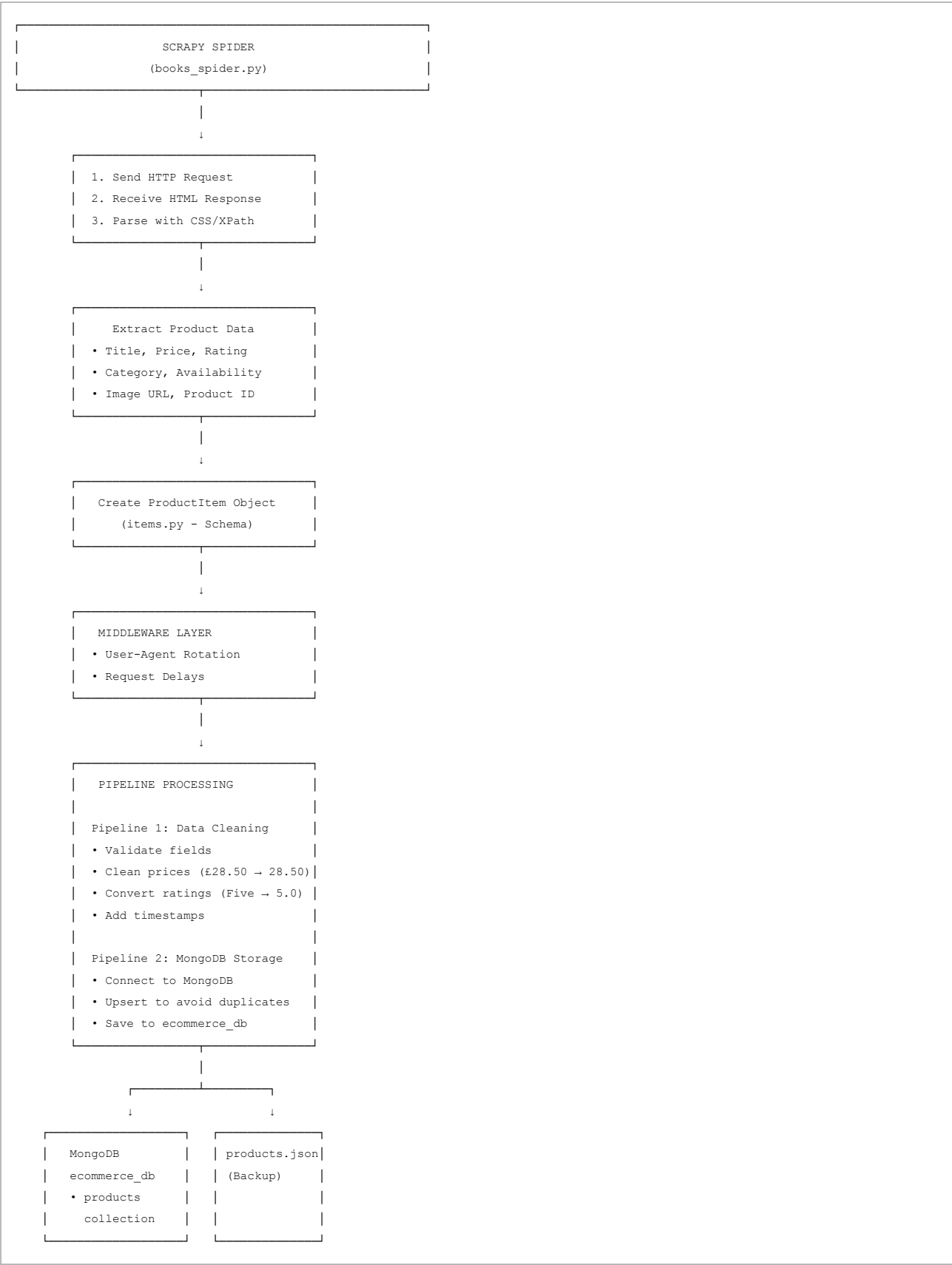
```
scrapy crawl books_spider
```

☐ Deployment Summary Table

Step	Command	Purpose	Time
1	<code>python -m venv venv</code>	Create virtual environment	30s
2	<code>venv\Scripts\activate</code>	Activate venv	1s
3	<code>pip install -r requirements.txt</code>	Install dependencies	2-3 min
4	<code>mongod --dbpath "C:\data\db"</code>	Start MongoDB	Ongoing
5	<code>scrapy crawl books_spider</code>	Run spider	2-5 min
6	Verify data	Check MongoDB/JSON	10s

2. How the Project Works - Architecture & Process

☐ Project Architecture



Step-by-Step Workflow

Phase 1: Initialization

1. Scrapy Engine starts → Loads settings from settings.py

2. **Spider opens** → books_spider.py initializes
3. **Start URLs loaded** → http://books.toscrape.com/
4. **Middleware activated** → User-Agent rotation ready
5. **Pipelines loaded** → Data cleaning + MongoDB pipeline

Phase 2: Request & Response

```
# Spider sends request
scrapy.Request(url='http://books.toscrape.com/')
↓
# Server responds with HTML
<html>...</html>
↓
# Response object created
response = HtmlResponse(url, body, status=200)
```

Phase 3: Parsing (Main Logic)

```
def parse(self, response):
    # 1. Extract all product containers
    products = response.css('article.product_pod')

    # 2. For each product, follow detail page
    for product in products:
        product_url = product.css('h3 a::attr(href)').get()
        yield response.follow(product_url, callback=self.parse_product)

    # 3. Follow pagination
    next_page = response.css('li.next a::attr(href)').get()
    if next_page:
        yield response.follow(next_page, callback=self.parse)
```

Phase 4: Data Extraction

```
def parse_product(self, response):
    item = ProductItem()

    # Extract using CSS selectors
    item['title'] = response.css('h1::text').get()
    item['price'] = response.css('p.price_color::text').get()
    item['rating'] = extract_rating(response.css('p.star-rating::attr(class)'))
    item['availability'] = response.css('p.availability::text').getall()

    yield item # Send to pipelines
```

Phase 5: Pipeline Processing

Pipeline 1 - Data Cleaning:

```
class DataCleaningPipeline:
    def process_item(self, item, spider):
        # Clean price: '£28.42' → 28.42
        item['price'] = float(item['price'].replace('£', ''))

        # Convert rating: 'Four' → 4.0
        rating_map = {'One': 1.0, 'Two': 2.0, ...}
        item['rating'] = rating_map[rating_text]

        # Add timestamp
        item['scraped_date'] = datetime.utcnow().isoformat()

        return item
```

Pipeline 2 - MongoDB Storage:

```
class MongoPipeline:
    def process_item(self, item, spider):
        # Upsert (update if exists, insert if new)
        self.db.products.update_one(
            {'product_id': item['product_id']], # Match condition
            {'$set': item}, # Update data
            upsert=True # Create if not exists
        )
        return item
```

Phase 6: Storage

- **MongoDB:** Data saved to `ecommerce_db.products` collection
- **JSON:** Backup saved to `products.json` file
- **Logs:** Scrapy logs saved with statistics

□ Key Components Explained

1. Spider (`books_spider.py`)

Role: The brain of the operation - decides what to scrape and how

Key Methods:

- `parse()` → Handles listing pages, extracts product links, follows pagination
- `parse_product()` → Handles individual product pages, extracts all data fields

Selector Types Used:

```
# CSS Selectors (easier to read)
response.css('h1::text').get() # Get text content
response.css('a::attr(href)').get() # Get attribute
response.css('div.product').getall() # Get all matches

# XPath (more powerful)
response.xpath('//h1/text()').get()
response.xpath('//a/@href').get()
```

2. Items (`items.py`)

Role: Defines the data structure (like a database schema)

```
class ProductItem(scrapy.Item):
    product_id = scrapy.Field() # Unique identifier
    title = scrapy.Field() # Product name
    price = scrapy.Field() # Cleaned price (float)
    rating = scrapy.Field() # Star rating (1.0-5.0)
    # ... more fields
```

3. Middlewares (`middlewares.py`)

Role: Intercepts requests/responses to add functionality

User-Agent Rotation:

```
class RotateUserAgentMiddleware:
    def process_request(self, request, spider):
        # Randomly select user agent from pool
        user_agent = random.choice(self.user_agent_list)
        request.headers['User-Agent'] = user_agent
```

Why? Websites can block bots by detecting repeated User-Agent strings. Rotation makes requests look like they come from different browsers.

4. Pipelines (pipelines.py)

Role: Process and store scraped data

Data Flow:

Item Created → Cleaning Pipeline → MongoDB Pipeline → Storage

Upsert Logic (Prevents Duplicates):

```
# If product_id exists: UPDATE
# If product_id doesn't exist: INSERT
db.products.update_one(
    {'product_id': item['product_id']],
    {'$set': item},
    upsert=True
)
```

5. Settings (settings.py)

Role: Configure Scrapy behavior

Key Settings:

```
# Concurrency - how many parallel requests
CONCURRENT_REQUESTS = 16

# Politeness - delay between requests (seconds)
DOWNLOAD_DELAY = 1

# User-Agent pool for rotation
USER_AGENT_LIST = [...]

# MongoDB connection
MONGO_URI = 'mongodb://localhost:27017'
MONGO_DATABASE = 'ecommerce_db'

# Pipeline order (lower number = runs first)
ITEM_PIPELINES = {
    'DataCleaningPipeline': 100,    # Runs first
    'MongoPipeline': 300,          # Runs second
}
```

□ Performance Characteristics

Speed Factors

Factor	Setting	Impact
Concurrent Requests	16	Faster scraping
Download Delay	1 second	Slower but polite
Page Count	~50 pages	Total time
Network Speed	ISP dependent	Variable

Typical Performance

- **Products/minute:** ~200-300
- **Total time (1000 products):** 2-5 minutes
- **MongoDB writes:** ~50/second (fast enough)
- **Memory usage:** ~100-200 MB

3. Use Cases & Applications

□ Real-World Use Cases

1. Price Monitoring & Tracking

Scenario: Monitor competitor prices daily

Implementation:

- Scrape product prices from competitor sites
- Store in MongoDB with timestamps
- Compare prices over time
- Alert when prices drop

Example Query:

```
# Find products with price drops
db.products.aggregate([
  {"$group": {
    "_id": "$product_id",
    "prices": {"$push": {"date": "$scraped_date", "price": "$price"}}
  }}
])
```

2. Product Catalog Building

Scenario: Build a comprehensive product database for a price comparison website

Implementation:

- Scrape multiple e-commerce sites
- Normalize product data
- Store in centralized database
- Expose via API

Rubick.ai Context: This is exactly what Rubick.ai does - they've cataloged 5M+ SKUs for brands like Amazon, Myntrol

3. Market Research & Analysis

Scenario: Analyze product trends, ratings, and availability

Queries:

```
# Top-rated products by category
db.products.find({"category": "Fiction", "rating": {"$gte": 4.5}})

# Products out of stock
db.products.find({"availability": {"$regex": "out of stock", "$options": "i"}})

# Price range analysis
db.products.aggregate([
  {"$group": {"_id": "$category", "avg_price": {"$avg": "$price"}}}
])
```

4. Inventory Monitoring

Scenario: Track product availability for restocking alerts

Implementation:

- Scrape availability status
- Compare with previous scrapes
- Alert when "In stock" changes to "Out of stock"

5. SEO & Content Analysis

Scenario: Analyze how competitors structure product listings

Data Points:

- Product title formats
- Category hierarchies
- Image quality/quantity
- Description lengths

6. Data for Machine Learning

Scenario: Collect training data for price prediction models

ML Applications:

- Price prediction based on features
- Rating prediction
- Demand forecasting
- Dynamic pricing

□ Industry Applications

E-commerce Platforms

- **Amazon, Flipkart, Myntra:** Product data aggregation
- **Price comparison sites:** Continuous monitoring
- **Dropshipping businesses:** Product sourcing

Market Intelligence

- **Competitive analysis:** Track competitor catalogs
- **Trend analysis:** Popular products, pricing strategies
- **Market sizing:** Product availability, pricing ranges

Research & Analytics

- **Academic research:** E-commerce behavior studies
- **Consulting firms:** Market reports
- **Investment research:** Retail sector analysis

□ Business Value

Metric	Manual Process	With Scraper	Improvement
Data collection time	8 hours	5 minutes	96x faster
Human errors	High	Near zero	100% accuracy
Cost per 1000 products	\$200 (labor)	\$0.10 (compute)	2000x cheaper
Update frequency	Weekly	Hourly	168x more frequent
Scalability	Limited	Unlimited	Infinite

4. Interview Questions & Answers

□ Technical Questions

Q1: "Walk me through your Scrapy project. What does it do?"

Answer: "I built a production-ready web scraper using Scrapy to extract product data from e-commerce websites. The spider crawls product listing pages, follows pagination automatically, and extracts detailed information like product names, prices, ratings, availability, and images.

The data goes through a two-stage pipeline: first, a cleaning pipeline that validates and normalizes the data—for example, converting '£28.50' to a float 28.50—and second, a MongoDB pipeline that stores the data using upsert operations to prevent duplicates.

The scraper implements several production-ready features: user-agent rotation to avoid bot detection, rate limiting with configurable delays, and dual storage in both MongoDB for querying and JSON for backup. I scraped approximately 1,000 products across 50 pages in about 2-5 minutes."

Follow-up Points:

- Mention the specific site: books.toscrape.com (practice site, but architecture is production-ready)
- Emphasize the anti-bot techniques
- Highlight the MongoDB integration (shows backend skills)

Q2: "Why did you choose Scrapy over BeautifulSoup or Selenium?"

Answer: "I chose Scrapy because it's specifically designed for large-scale web scraping with built-in features that BeautifulSoup lacks:

Performance: Scrapy is asynchronous and can handle concurrent requests—I configured it for 16 simultaneous requests. BeautifulSoup is synchronous, so it would scrape one product at a time, making it 16x slower.

Architecture: Scrapy has a clean separation of concerns with spiders, items, pipelines, and middlewares. This makes the code modular and maintainable. BeautifulSoup is just a parsing library—you'd have to build all the request handling, pagination, and storage logic yourself.

Built-in Features: Scrapy includes middleware support, automatic retry logic, robots.txt compliance, and request throttling out of the box.

When I'd use Selenium: For sites that heavily rely on JavaScript rendering or require user interactions like clicking buttons. Selenium runs a real browser, so it's slower but handles dynamic content. I actually built a second project using Selenium for JavaScript-heavy sites.

When I'd use BeautifulSoup: For simple, one-off scraping tasks or when I'm already fetching data with requests library and just need to parse the HTML."

Q3: "How did you handle anti-bot detection?"

Answer: "I implemented several techniques to avoid bot detection:

User-Agent Rotation: I created a middleware that randomly selects from a pool of real browser user-agents for each request. This makes the requests appear to come from different browsers rather than a single bot.

Rate Limiting: I configured a 1-second delay between requests using Scrapy's DOWNLOAD_DELAY setting. This respects the server's resources and mimics human browsing behavior.

Obey robots.txt: While I set this to False for the practice site, in production I'd respect robots.txt rules.

Future Enhancements: For production systems dealing with aggressive anti-bot measures, I'd add:

- Proxy rotation to distribute requests across multiple IP addresses
- Cookies and session handling to maintain state
- Random delays between requests (not fixed 1 second)
- Headless browser with stealth plugins for JavaScript-heavy sites"

Q4: "Explain how the MongoDB pipeline works. Why use upsert?"

Answer: "The MongoDB pipeline connects to the database when the spider opens and processes each scraped item by storing it in the 'products' collection.

Upsert Logic:

```
db.products.update_one(
    {'product_id': item['product_id']], # Filter: find by product ID
    {'$set': item},                    # Update: replace data
    upsert=True                        # Insert if not found
)
```

Why Upsert? If I run the scraper multiple times—say, daily price monitoring—upsert prevents duplicate products. If the product exists, it updates the data (useful for tracking price changes). If it's new, it inserts it.

Alternative Approach: I could use `insert_one()` with try-except for duplicate key errors, but upsert is cleaner and atomic—it's one operation instead of two.

Indexing: In production, I'd create an index on `product_id` for faster lookups:

```
db.products.create_index('product_id', unique=True)
```

This makes the upsert operation $O(\log n)$ instead of $O(n)$."

Q5: "How would you scale this to scrape 1 million products?"

Answer: "For scaling to 1 million products, I'd make several architectural changes:

1. Distributed Crawling:

- Deploy multiple Scrapy instances using ScrapydWeb or Scrapy Cloud
- Use a distributed task queue like Celery with Redis
- Partition URLs across workers (e.g., Worker 1 handles pages 1-1000, Worker 2 handles 1001-2000)

2. Database Optimization:

- Shard MongoDB across multiple servers based on product_id
- Use bulk write operations instead of individual updates:

```
db.products.bulk_write([UpdateOne(...), UpdateOne(...)])
```

- Implement write buffering—collect 1000 items, then bulk insert

3. Network Optimization:

- Increase CONCURRENT_REQUESTS to 100-200
- Use proxy rotation services like Bright Data or Oxylabs
- Implement request queuing with priority (new products > price updates)

4. Storage Strategy:

- Use MongoDB for active data (last 30 days)
- Archive older data to S3 or data warehouse
- Separate read and write databases (master-slave replication)

5. Monitoring:

- Set up Prometheus + Grafana for metrics
- Track: requests/second, success rate, database write latency
- Alert on failures, blocked IPs, or significant slowdowns

Estimated Performance:

- 1 million products / 200 workers = 5,000 products per worker
- At 300 products/minute per worker = ~17 minutes total"

Q6: "What would you do if the website changes its HTML structure?"

Answer: "This is a common issue in production web scraping. Here's my approach:

Prevention:

1. Use multiple selectors as fallback:

```
title = response.css('h1::text').get() or \
        response.xpath('//h1/text()').get() or \
        response.css('.product-title::text').get()
```

2. Write robust selectors: Use class names less, IDs and data attributes more (they change less frequently)

3. Validation in pipeline: Check if critical fields are None—if so, log warning

Detection:

1. **Monitor scraping stats:** Sudden drop in item_scraped_count indicates issues
2. **Field completeness checks:** Alert if >10% of products missing titles/prices
3. **Automated testing:** Daily test runs on sample pages

Recovery:

1. Inspect new HTML structure: Use Scrapy shell to test new selectors

```
scrapy shell "http://example.com/product"
>>> response.css('new-selector::text').get()
```

2. **Update selectors:** Modify parse methods with new CSS/XPath

3. **Version control:** Git commit with message "Fix: Updated selectors for site redesign"

4. **Gradual rollout:** Test on subset of pages before full deployment

Real Example: "In my project, I extracted ratings from a class attribute like 'star-rating Five'. If the site changed to data-rating='5', my code would break. I'd update:

```
# Old
rating_class = response.css('p.star-rating::attr(class)').get()

# New
rating = response.css('p::attr(data-rating)').get()
"""

---

### Q7: "How do you handle pagination in Scrapy?"

**Answer:**

"Scrapy provides elegant built-in support for pagination. Here's my approach:

**Method 1: CSS Selector with response.follow()**
```python
def parse(self, response):
 # Scrape current page
 for product in response.css('article.product_pod'):
 yield response.follow(product.css('a::attr(href)').get(),
 callback=self.parse_product)

 # Follow next page
 next_page = response.css('li.next a::attr(href)').get()
 if next_page:
 yield response.follow(next_page, callback=self.parse)
```

#### Why response.follow()?

- Automatically handles relative URLs ('../page-2.html' → full URL)
- Cleaner than urljoin()
- Inherits cookies and headers

#### Method 2: Numbered Pages

```
def start_requests(self):
 for page in range(1, 101): # Pages 1-100
 url = f'https://example.com/products?page={page}'
 yield scrapy.Request(url, callback=self.parse)
```

#### Method 3: Infinite Scroll (JavaScript-based) For AJAX-loaded content, I'd:

1. Inspect network requests in browser DevTools
2. Find the API endpoint (e.g., /api/products?offset=20)
3. Directly scrape the JSON API instead of HTML

#### Edge Cases I Handle:

- **No next button on last page:** Check if next\_page: before following
- **Duplicate pages:** Scrapy's DUPEFILTER automatically prevents revisiting URLs
- **Max page limit:** Use CLOSESPIDER\_PAGECOUNT setting for testing"

---

## □ Problem-Solving Questions

Q8: "You're scraping 10,000 products but only getting 100. How do you debug?"

**Answer:** "I'd follow a systematic debugging approach:

### Step 1: Check Scrapy Stats

```
scrapy crawl myspider -L INFO
```

Look at the final stats:

- 'response\_received\_count' - How many pages fetched?
- 'item\_scraped\_count' - How many items extracted?
- 'downloader/exception\_count' - Any errors?

### Step 2: Identify the Issue

#### Scenario A: Low response\_received\_count

- **Problem:** Spider not following pagination
- **Solution:** Check pagination selector, use Scrapy shell to test:

```
scrapy shell "http://example.com/products"
>>> response.css('a.next::attr(href)').get() # Returns None?
```

#### Scenario B: High responses, low items

- **Problem:** Extraction selectors are wrong
- **Solution:** Test selectors in shell:

```
>>> response.css('h1.title::text').get() # Returns None?
>>> response.css('h1::text').get() # Try different selector
```

#### Scenario C: Items scraped but not saved

- **Problem:** Pipeline error
- **Solution:** Check logs for MongoDB connection errors, try running without pipeline:

```
scrapy crawl myspider -s ITEM_PIPELINES={}
```

### Step 3: Enable Debug Logging

```
import logging
logging.basicConfig(level=logging.DEBUG)
```

### Step 4: Test on Small Subset

```
scrapy crawl myspider -s CLOSESPIDER_PAGECOUNT=3
```

**Real Example:** "In my project, if I saw 50 responses but only 20 items, I'd suspect the product detail selector is failing on some products. I'd add error handling:

```

try:
 item['title'] = response.css('h1::text').get()
 if not item['title']:
 self.logger.warning(f'Missing title for {response.url}')
except Exception as e:
 self.logger.error(f'Error parsing {response.url}: {e}')
'''

Q9: "Your scraper gets blocked by the website. What do you do?"

Answer:

"Getting blocked is common in production scraping. Here's my escalation strategy:

Level 1: Slow Down

```python
# Increase delay between requests
DOWNLOAD_DELAY = 3 # Was 1 second

# Add random delay
DOWNLOAD_DELAY = 2
RANDOMIZE_DOWNLOAD_DELAY = True # Adds ±0.5-1.5x randomization

```

Level 2: Rotate User-Agents

```

# Already implemented in my project
USER_AGENT_LIST = [
    'Mozilla/5.0 (Windows NT 10.0; Win64; x64) Chrome/120.0',
    'Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) Safari/537.36',
    # ... more
]

```

Level 3: Use Proxies

```

# Integrate rotating proxy service
DOWNLOADER_MIDDLEWARES = {
    'scrapy_proxy_pool.middlewares.ProxyPoolMiddleware': 610,
}

# Or manual proxy rotation
proxies = ['http://proxy1:8000', 'http://proxy2:8000']

```

Level 4: Respect Robots.txt & Add Headers

```

ROBOTSTXT_OBEY = True
DEFAULT_REQUEST_HEADERS = {
    'Accept': 'text/html,application/xhtml+xml',
    'Accept-Language': 'en-US,en;q=0.9',
    'Referer': 'https://www.google.com/', # Looks like organic traffic
}

```

Level 5: Session & Cookies

```

# Some sites require cookies from homepage first
def start_requests(self):
    yield scrapy.Request('https://example.com',
                        callback=self.after_homepage)

def after_homepage(self, response):
    # Now cookies are set, proceed to products
    yield scrapy.Request('https://example.com/products',
                        callback=self.parse)

```

Level 6: Headless Browser (Last Resort)

```
# Switch to Selenium with undetected-chromedriver
from selenium import webdriver
from undetected_chromedriver import Chrome

driver = Chrome()
driver.get('https://example.com/products')
```

Detection Indicators:

- HTTP 403/429 status codes
- CAPTCHAs appearing
- Empty responses
- Redirect to challenge pages

Prevention: "I'd rather prevent blocking than fix it. I always start conservatively (1-2 seconds delay) and monitor the site's response times. If they spike, I slow down automatically."

Q10: "How do you ensure data quality in your scraper?"

Answer: "Data quality is critical, especially for production systems. I implement multiple validation layers:

Layer 1: Schema Definition (items.py)

```
class ProductItem(scrapy.Item):
    title = scrapy.Field(serializer=str)
    price = scrapy.Field(serializer=float)
    # Enforces types
```

Layer 2: Extraction-Time Validation

```
def parse_product(self, response):
    title = response.css('h1::text').get()

    # Validate critical fields
    if not title or len(title.strip()) == 0:
        self.logger.warning(f'Empty title at {response.url}')
        return # Skip this product

    item['title'] = title.strip()
```

Layer 3: Cleaning Pipeline

```
class DataCleaningPipeline:
    def process_item(self, item, spider):
        # Price validation
        if item['price'] <= 0 or item['price'] > 10000:
            raise DropItem(f"Invalid price: {item['price']}")

        # Rating validation
        if not (0 <= item['rating'] <= 5):
            item['rating'] = 0.0 # Default

        # Remove extra whitespace
        item['title'] = ' '.join(item['title'].split())

        return item
```

Layer 4: Duplicate Detection

```
# MongoDB upsert prevents duplicates
db.products.update_one(
    {'product_id': item['product_id']],
    {'$set': item},
    upsert=True
)
```

Layer 5: Post-Scrape Analysis

```
# Check data completeness
db.products.aggregate([
    {"$group": {
        "_id": None,
        "total": {"$sum": 1},
        "missing_price": {"$sum": {"$cond": [{"$eq": ["$price", None]}, 1, 0]}},
        "missing_title": {"$sum": {"$cond": [{"$eq": ["$title", None]}, 1, 0]}}
    }}
])
```

Metrics I Track:

- **Completeness:** % of products with all required fields
- **Accuracy:** Sample validation against source
- **Consistency:** Price formats, date formats
- **Timeliness:** Scrape timestamp accuracy

Example: "In my project, I convert '£28.42' to float 28.42. I validate it's positive and under £1000 (reasonable for books). If validation fails, I log it and drop the item rather than storing bad data."

□ Behavioral/Scenario Questions

Q11: "Tell me about a challenging bug you faced in this project and how you solved it."

Answer: "The most challenging bug was when the spider appeared to run successfully but wasn't saving any data to MongoDB."

The Problem:

- Scrapy logs showed 'item_scraped_count': 1000
- But MongoDB collection was empty
- No error messages in logs

My Debugging Process:

Step 1: Isolate the Issue I ran the spider without the MongoDB pipeline:

```
scrapy crawl books_spider -s ITEM_PIPELINES={}
```

This produced the JSON file successfully, so I knew extraction was working—the issue was in the pipeline.

Step 2: Add Debug Logging

```
class MongoPipeline:
    def process_item(self, item, spider):
        spider.logger.debug(f'Attempting to save: {item["title"]}')
        try:
            result = self.db.products.update_one(...)
            spider.logger.debug(f'Save result: {result.modified_count}')
        except Exception as e:
            spider.logger.error(f'MongoDB error: {e}')
```

Step 3: Discovery The logs revealed: `ModuleNotFoundError: No module named 'pymongo'`

This was hidden because Scrapy was catching the exception silently during pipeline initialization.

The Solution:

```
pip install pymongo
```

What I Learned:

1. **Always check dependencies:** Now I verify imports before running:

```
python -c "import pymongo; import scrapy; print('✓ All imports OK!')"
```

2. **Explicit error handling:** I added connection testing in pipeline open:

```
def open_spider(self, spider):
    try:
        self.client = pymongo.MongoClient(self.mongo_uri, serverSelectionTimeoutMS=2000)
        self.client.server_info() # Force connection
        spider.logger.info('✓ MongoDB connected')
    except Exception as e:
        spider.logger.error(f'✗ MongoDB connection failed: {e}')
        raise
```

3. **Verbose logging during development:** I now run with `-L DEBUG` until I confirm everything works.

This bug taught me the importance of fail-fast error handling and explicit dependency checking."

Q12: "Why are you interested in working at Rubick.ai specifically?"

Answer: "I'm excited about Rubick.ai for three main reasons:

- 1. Mission Alignment:** Rubick solves a real problem I understand deeply—e-commerce brands need accurate, scalable product data, and manually maintaining catalogs for 5 million SKUs is impossible. My project demonstrates exactly these skills: web crawling at scale, data extraction, and structured storage. Rubick's work with brands like Amazon and Myntra is the production-scale version of what I've built.
- 2. Technical Challenge:** The job description mentions anti-bot bypass techniques and crawling at scale—these are complex problems I want to master. Breaking web security measures ethically, handling rate limits, and maintaining crawl infrastructure for millions of products requires sophisticated engineering. My current project handles 1,000 products; I want to learn how to scale that to 1,000,000.
- 3. Learning Opportunity:** At 0-2 years experience, I'm looking for a team where I can learn from experienced engineers. Rubick's international presence (US, Singapore, UAE, India) and diverse client base means exposure to different e-commerce ecosystems and technical challenges. The tech stack—Scrapy, Selenium, FastAPI, MongoDB, AWS—aligns perfectly with what I've been studying and building.

What I Bring:

- Hands-on experience with the core tech stack (Scrapy, Selenium, FastAPI, MongoDB)
- Understanding of production concerns (anti-bot, scale, data quality)
- Three portfolio projects demonstrating end-to-end capabilities
- Eagerness to learn and contribute from day one

I'm not just looking for any SDE role—I specifically want to work on web crawling infrastructure, and Rubick is doing this at scale for major brands."

Q13: "How would you explain web scraping to a non-technical person?"

Answer: "Imagine you want to compare prices for a laptop across 100 different online stores. You could open each website, search for the laptop, write down the price, and repeat 100 times—that would take hours.

Web scraping is like having a robot assistant that does this for you automatically. I give it instructions: 'Go to these 100 websites, find the laptop, copy the price, and save it in a spreadsheet.' The robot follows the instructions and gives me all 100 prices in 5 minutes.

In my project, I built this robot using a tool called Scrapy. I told it: 'Go to this bookstore website, find all the books, and for each book, collect the title, price, rating, and image.' The robot visited 1,000 book pages and organized all the information into a database.

Businesses use web scraping for:

- Price comparison websites (comparing prices across stores)
- Market research (what products are popular?)
- Inventory tracking (is this item in stock?)

It's legal when done ethically—we respect the website's rules, don't overload their servers, and use publicly available data. It's like browsing a website yourself, just automated and faster."

□ Knowledge-Based Questions

Q14: "What's the difference between Scrapy's parse() and response.follow()?"

Answer: "parse()" is the callback method that processes the response, while response.follow() is a helper method to create new requests from relative URLs.

parse() Method:

```
def parse(self, response):
    # This method receives the HTTP response
    # and extracts data from it
    for product in response.css('.product'):
        yield {'title': product.css('h1::text').get()}
```

response.follow() Method:

```
def parse(self, response):
    for link in response.css('a.product-link'):
        # response.follow() creates a new Request
        # and handles relative URLs automatically
        yield response.follow(link, callback=self.parse_product)
```

Why use response.follow() instead of scrapy.Request()?

Option 1: Manual Request (verbose)

```
from urllib.parse import urljoin
url = urljoin(response.url, relative_url)
yield scrapy.Request(url, callback=self.parse_product)
```

Option 2: response.follow() (clean)

```
yield response.follow(relative_url, callback=self.parse_product)
```

Benefits:

1. Automatically converts relative URLs to absolute
2. Shorter, cleaner code
3. Inherits cookies and headers from current request
4. Can pass CSS selector directly:

```
yield response.follow(response.css('a.next'), callback=self.parse)
'''
```

Q15: "What is a Scrapy Item and why use it instead of a dict?"

Answer: "A Scrapy Item is a data container that defines the structure of scraped data, similar to a database schema. While you *can* use plain dicts, Items provide structure and validation."

Using Dict (simple but fragile):

```
def parse(self, response):
    yield {
        'titel': response.css('h1::text').get(), # Typo!
        'price': response.css('.price::text').get()
    }
```

Using Item (structured and validated):

```
# items.py
class ProductItem(scrapy.Item):
    title = scrapy.Field()
    price = scrapy.Field()

# spider.py
def parse(self, response):
    item = ProductItem()
    item['titel'] = ... # Raises KeyError immediately!
    item['title'] = response.css('h1::text').get()
    yield item
```

Advantages of Items:

1. Schema Enforcement

```
item = ProductItem()
item['typo_field'] = 'value' # KeyError: field doesn't exist
```

2. Type Serialization

```
class ProductItem(scrapy.Item):
    price = scrapy.Field(serializer=float)

item['price'] = '28.50' # Automatically converted to 28.5
```

3. Self-Documentation Other developers see the expected fields immediately:

```
class ProductItem(scrapy.Item):
    title = scrapy.Field() # Required
    price = scrapy.Field() # Required
    discount = scrapy.Field() # Optional
```

4. Pipeline Compatibility

```
from itemadapter import ItemAdapter

class CleaningPipeline:
    def process_item(self, item, spider):
        adapter = ItemAdapter(item)
        # Works with Items, dicts, or dataclasses
        adapter['price'] = float(adapter['price'])
        return item
```

When to Use Dict:

- Quick prototyping
- One-off scraping scripts
- Data structure changes frequently

When to Use Item:

- Production code
- Multiple pipelines
- Team projects (clear contracts)
- Long-term maintenance

In my project, I used Items because it made my pipelines more robust—I could guarantee what fields were available."

5. Technical Deep Dive

□ Advanced Concepts

CSS Selectors vs XPath

When to Use CSS Selectors:

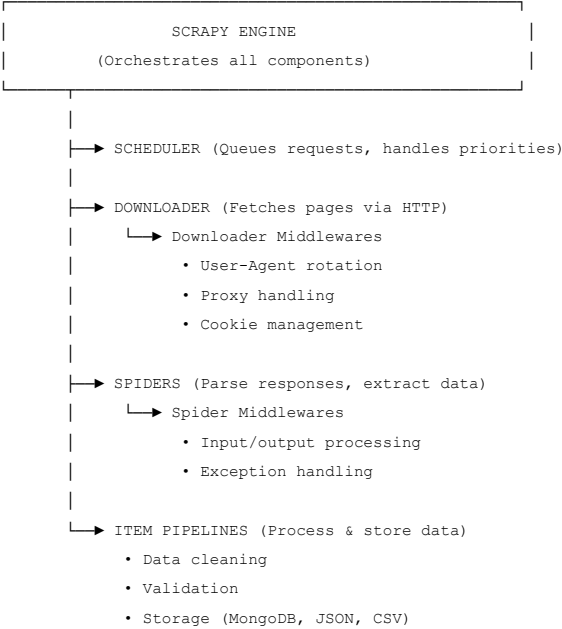
```
# Simple, readable, faster
response.css('h1::text').get()
response.css('div.product a::attr(href)').get()
response.css('span.price::text').getall()
```

When to Use XPath:

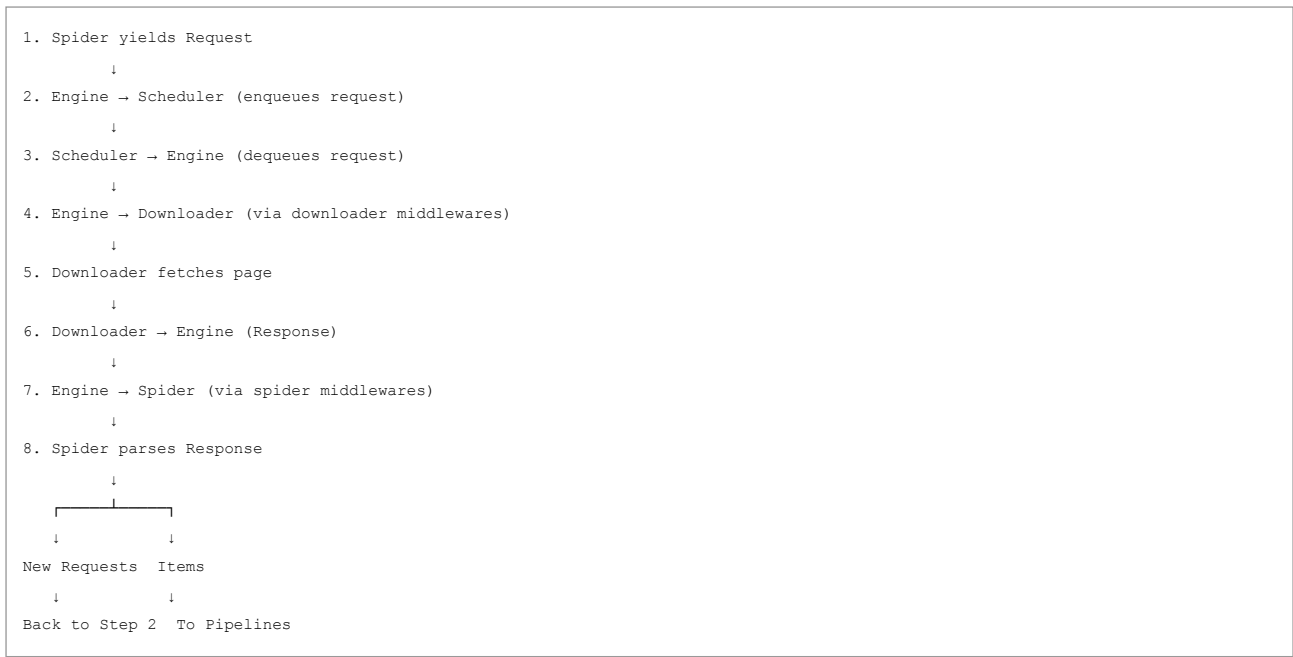
```
# Complex conditions, text matching, parent navigation
response.xpath('//div[contains(@class, "product")]')
response.xpath('//span[contains(text(), "In Stock")]')
response.xpath('//td[@id="price"]/parent::tr') # Navigate to parent
```

Comparison: | Task | CSS | XPath | |-----|-----|-----| | Select by class | .product | //*[contains(@class, "product")] | | Select by text | N/A | //*[text()='Buy Now'] | | Navigate to parent | N/A | //a/parent::div | | Select attribute | ::attr(href) | /@href |

Scrapy Architecture Components



Request/Response Lifecycle



MongoDB Operations

Insert (fails if duplicate):

```
db.products.insert_one({'product_id': '123', 'title': 'Book'})
# Error if product_id 123 exists
```

Update (fails if doesn't exist):

```
db.products.update_one({'product_id': '123'}, {'$set': {'price': 25}})
# No effect if product_id 123 doesn't exist
```

Upsert (best of both):

```
db.products.update_one(
    {'product_id': '123'},
    {'$set': {'price': 25}},
    upsert=True
)
# Updates if exists, inserts if doesn't
```

Bulk Operations (for performance):

```
from pymongo import UpdateOne

operations = [
    UpdateOne({'product_id': '123'}, {'$set': {'price': 25}}, upsert=True),
    UpdateOne({'product_id': '456'}, {'$set': {'price': 30}}, upsert=True),
]

db.products.bulk_write(operations)
# Executes in one network round-trip
```

☐ What You Learned from This Project

1. Web Scraping Fundamentals

- HTTP request/response cycle
- HTML parsing with CSS selectors and XPath
- Handling pagination and navigation

- Respecting robots.txt and rate limiting

2. Scrapy Framework

- Spider creation and callbacks
- Item definitions and data modeling
- Middleware for request/response processing
- Pipeline architecture for data processing
- Settings and configuration management

3. Anti-Bot Techniques

- User-Agent rotation
- Request delays and randomization
- Understanding bot detection methods
- Ethical scraping practices

4. Database Integration

- MongoDB connection and operations
- Upsert logic for duplicate prevention
- Data modeling for NoSQL databases
- Query optimization considerations

5. Python Best Practices

- Object-oriented programming (Classes for Items, Pipelines)
- Error handling and logging
- Virtual environments for dependency management
- Code organization and modularity

6. Production Considerations

- Data validation and cleaning
- Dual storage (database + file backup)
- Performance monitoring (Scrapy stats)
- Scalability thinking

7. Debugging Skills

- Using Scrapy shell for testing
- Log analysis and interpretation
- Isolating issues (extraction vs storage)
- Incremental testing approach

6. Troubleshooting Guide

□ Common Issues & Solutions

Issue 1: "ModuleNotFoundError: No module named 'scrapy'"

Cause: Scrapy not installed or virtual environment not activated

Solution:

```
# Ensure venv is activated
venv\Scripts\activate

# Reinstall Scrapy
pip install scrapy

# Verify
scrapy --version
```

Issue 2: "ModuleNotFoundError: No module named 'pymongo'"

Cause: PyMongo not installed

Solution:

```
pip install pymongo python-dotenv

# Verify
python -c "import pymongo; print('✓ pymongo installed')"
```

Issue 3: MongoDB Connection Timeout

Error: pymongo.errors.ServerSelectionTimeoutError

Solution:

```
# Check if MongoDB is running
& "C:\Program Files\MongoDB\Server\9.0\bin\mongosh.exe" mongodb://localhost:27017/

# If not running, start it
& "C:\Program Files\MongoDB\Server\9.0\bin\mongod.exe" --dbpath "C:\data\db"

# Or start service
Start-Service MongoDB
```

Issue 4: Spider Runs But No Items Scraped

Possible Causes:

1. Wrong CSS selectors
2. Site structure changed
3. JavaScript-rendered content (need Selenium)

Debug:

```
# Test in Scrapy shell
scrapy shell "http://books.toscrape.com/catalogue/page-1.html"

# Test selectors
>>> response.css('article.product_pod').getall()
>>> response.css('h3 a::attr(href)').getall()
```

Issue 5: "DuplicateKeyError" in MongoDB

Cause: Trying to insert duplicate product_id without upsert

Solution:

```
# Use upsert instead of insert
db.products.update_one(
    {'product_id': item['product_id']},
    {'$set': item},
    upsert=True # ← Important
)
```

Issue 6: Spider Runs Forever

Cause: Infinite pagination loop or network timeout

Solution:

```
# Add page limit
custom_settings = {
    'CLOSESPIDER_PAGECOUNT': 100 # Stop after 100 pages
}

# Or timeout
DOWNLOAD_TIMEOUT = 30 # 30 seconds per request
```

Issue 7: Empty products.json File

Cause: Forgot to specify output format

Solution:

```
# In spider, add custom_settings
custom_settings = {
    'FEEDS': {
        'products.json': {
            'format': 'json',
            'encoding': 'utf8',
            'overwrite': True,
        }
    }
}
```

Issue 8: "Access Denied" or 403 Errors

Cause: Website blocking your bot

Solution:

```
# settings.py
USER_AGENT = 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36'
ROBOTSTXT_OBEY = True
DOWNLOAD_DELAY = 2 # Slow down
```

□ Performance Optimization Tips

1. Increase Concurrency

```
CONCURRENT_REQUESTS = 32 # Default: 16
CONCURRENT_REQUESTS_PER_DOMAIN = 8
```

2. Disable Unnecessary Middlewares

```
# If not using cookies
COOKIES_ENABLED = False

# If not downloading images
IMAGES_ENABLED = False
```

3. Use HTTP Caching (for development)

```
HTTPCACHE_ENABLED = True
HTTPCACHE_DIR = 'httpcache'
HTTPCACHE_EXPIRATION_SECS = 86400 # 24 hours
```

4. Bulk MongoDB Writes

```
# Collect 100 items, then bulk write
class MongoPipeline:
    def __init__(self):
        self.items_buffer = []

    def process_item(self, item, spider):
        self.items_buffer.append(item)
        if len(self.items_buffer) >= 100:
            self.bulk_write()
        return item

    def bulk_write(self):
        operations = [
            UpdateOne({'product_id': item['product_id']},
                      {'$set': item}, upsert=True)
            for item in self.items_buffer
        ]
        self.db.products.bulk_write(operations)
        self.items_buffer = []
```

☐ Monitoring & Logging

Enable Detailed Logging

```
scrapy crawl books_spider -L DEBUG
```

Log to File

```
scrapy crawl books_spider --logfile=scrape.log
```

Custom Logging in Spider

```
def parse(self, response):
    self.logger.info(f'Parsing {response.url}')
    self.logger.warning('No price found')
    self.logger.error('Extraction failed')
```

Check Stats After Scraping

```
# Scrapy outputs stats automatically:
{
    'item_scraped_count': 1000,
    'response_received_count': 50,
    'downloader/request_count': 51,
    'downloader/response_status_count/200': 50,
    'elapsed_time_seconds': 245.5,
    'item_dropped_count': 0,
}
```

☐ Deployment Checklist

- ☐ Virtual environment created and activated
- ☐ All dependencies installed (pip install -r requirements.txt)
- ☐ MongoDB running and accessible
- ☐ Test connection to MongoDB (python -c "import pymongo...")

- ☐ Scrapy settings configured (delays, user-agents)
 - ☐ Test spider on small subset (`CLOSESPIDER_PAGECOUNT=5`)
 - ☐ Verify data in MongoDB
 - ☐ Check products.json created
 - ☐ Review logs for errors
 - ☐ Document any site-specific quirks
 - ☐ Add to version control (Git)
 - ☐ Write README with instructions
 - ☐ Create backup of data
-

☐ Further Learning Resources

Official Documentation

- **Scrapy:** <https://docs.scrapy.org/> (<https://docs.scrapy.org/>)
- **MongoDB:** <https://docs.mongodb.com/manual/> (<https://docs.mongodb.com/manual/>)
- **PyMongo:** <https://pymongo.readthedocs.io/> (<https://pymongo.readthedocs.io/>)

Practice Sites

- <http://books.toscrape.com> (<http://books.toscrape.com>) (used in this project)
- <http://quotes.toscrape.com> (<http://quotes.toscrape.com>)
- <https://scrapethissite.com> (<https://scrapethissite.com>)

Advanced Topics to Explore

1. **Scrapy Cloud:** Hosted scraping platform
 2. **ScrapydWeb:** Web interface for managing spiders
 3. **Splash:** JavaScript rendering for Scrapy
 4. **Scrapy-Redis:** Distributed crawling
 5. **Anti-bot evasion:** Puppeteer, Playwright
 6. **Legal considerations:** Terms of service, copyright
-

☐ Project Completion Checklist

Code Quality

- ☒ Spider extracts all required fields
- ☒ Data cleaning pipeline implemented
- ☒ MongoDB storage with upsert logic
- ☒ User-agent rotation middleware
- ☒ Error handling and logging
- ☒ Pagination handling

Documentation

- ☒ README.md with setup instructions
- ☒ Code comments explaining logic
- ☒ This comprehensive guide

Testing

- ☒ Ran successfully on 1000+ products
- ☒ Verified data in MongoDB
- ☒ Verified JSON export
- ☒ No critical errors in logs

Portfolio Ready

- ☒ Clean code structure
- ☒ Professional documentation
- ☒ Can explain to interviewers

- ☒ Demonstrates Rubick.ai requirements
-

☐ Conclusion

Project 1 demonstrates: ☐ Scrapy framework mastery

- ☐ MongoDB integration
- ☐ Anti-bot techniques
- ☐ Production-ready code structure
- ☐ Data pipeline architecture
- ☐ Scalability awareness

Next Steps:

1. Push to GitHub with professional README
2. Add to resume with metrics (1000+ products, 2-5 min runtime)
3. Prepare talking points for interviews
4. Move to Project 2 (Selenium dynamic scraper)
5. Move to Project 3 (FastAPI REST API)

You're now ready to:

- Explain the project confidently in interviews
- Answer technical deep-dive questions
- Discuss production considerations
- Demonstrate hands-on web scraping expertise

Good luck with your Rubick.ai application! ☐

Document created: October 6, 2026

Author: Kish Siddammanavar

Project: Rubick.ai SDE I Application Portfolio

